



ORACLE

MySQL Implementation Essentials Bootcamp

Database Security

Marco Carlessi (marco.carlessi@oracle.com)
Andreea Ciobotaru (andreea.Ciobotaru@oracle.com)
MySQL Sales Consulting

2023



Program Agenda

- Introduction
- Oracle MySQL Enterprise
- Oracle Cloud Infrastructure overview
- Installation and Architecture
- Database Design
- **Security**
- Backup
- Replication
- High Availability
- MySQL Enterprise Monitor
- HeatWave



93%

Security breaches are
preventable

--Online Trust Alliance

Regulatory Compliance

- Regulations

- GDPR, General Data Protection Regulation: Protection of Personal Data
- PCI – DSS: Payment Card Data
- HIPAA: Privacy of Health Data
- Sarbanes Oxley, GLBA, The USA Patriot Act:
- Financial Data, NPI "personally identifiable financial information"
- FERPA – Student Data
- Data Protection Act (UK): Protection of Personal Data

- Requirements

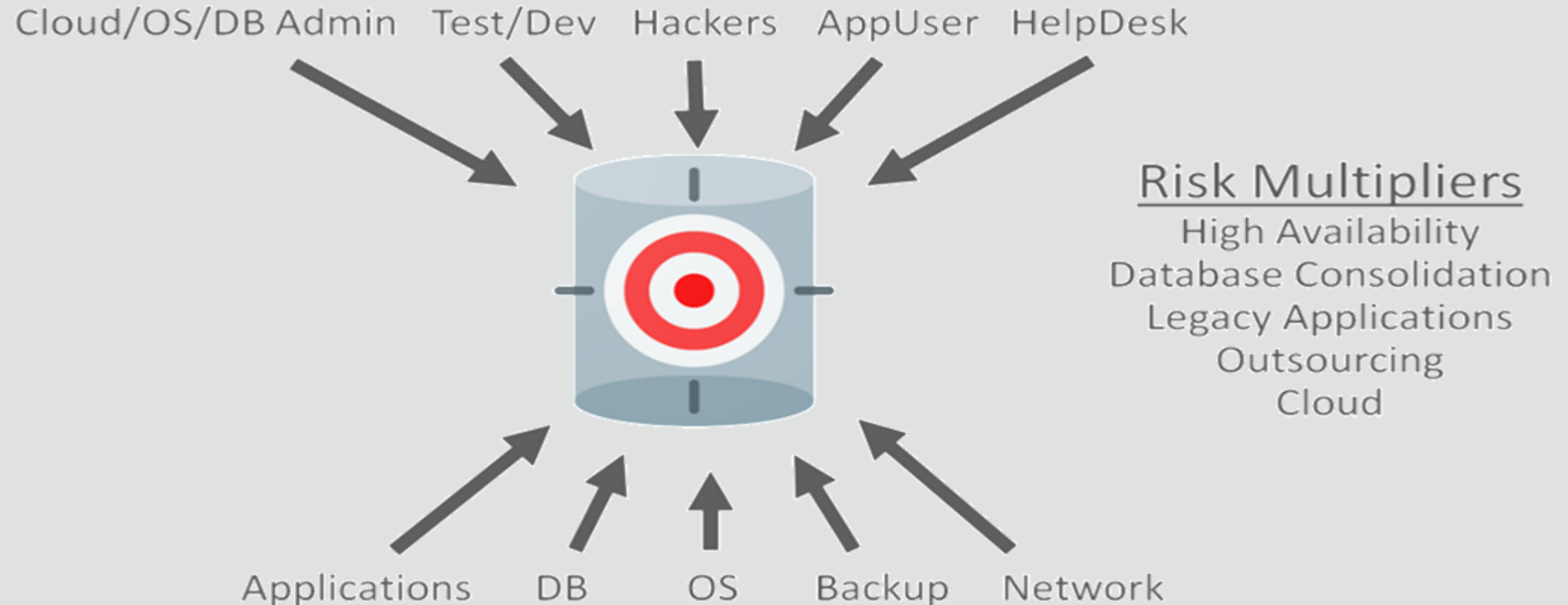
- Continuous Monitoring (Users, Schema, Backups, etc.)
- Data Protection (Encryption, Privilege Management, etc.)
- Data Retention (Backups, User Activity, etc.)
- Data Auditing (User activity, etc.)



Data Protection Act 1998

Attack Vectors and Targets for Databases

Complexity grows, Risk Grows



Database Vulnerabilities



- **Poor Configurations**
 - Prevent: Set controls and change default setting
- **Over Privileged Accounts**
 - Prevent: Use privilege policies
- **Weak Access Control**
 - Prevent: use dedicated administrative accounts
- **Weak Authentication**
 - Prevent: use strong password enforcement
- **Weak Auditing**
 - Prevent: enforce compliance & audit policies
- **Lack of Encryption**
 - Prevent: use data, backup, & network encryption
- **Proper Credential & Key Management**
 - Prevent: protect passwords, use key vaults
- **Unsecured Backups**
 - Prevent: encrypt backups
- **No Monitoring**
 - Prevent: implement security monitoring for users & objects
- **Poorly Coded Applications**
 - Use database firewall

Database Attacks



- **SQL Injection**
 - Prevention: DB Firewall (allow list), Input Validation
- **Buffer Overflow**
 - Prevention: Frequently apply Database Software updates, DB Firewall (allow list), Input Validation
- **Insider Abuse**
 - Prevention: Tight Access Controls, User specific authentication (no general accounts), Auditing, Monitoring, Encryption
- **Brute Force Attack**
 - Prevention: lock out accounts after a defined number of incorrect attempts
- **Network Eavesdropping**
 - Prevention: Require SSL/TLS for all Connections and Transport
- **Malware**
 - Prevention: Tight Access Controls, Limited Network IP access, Change default settings, Encryption

Database Malicious Actions



- **Information Disclosure:** obtain credit card and other personal information
 - Prevent: Encryption – Data and Network, implement Tighter Access Controls
- **Denial of Service:** run resource intensive queries
 - Prevent: Resource Usage Limits – Set various limits, e.g. Max Connections, Sessions, Timeouts, ...
- **Elevation of Privilege:** retrieve and use administrator credentials
 - Prevent: Stronger authentication, Access Controls, Auditing
- **Spoofing:** retrieve and use other credentials
 - Prevent: Stronger account and password policies
- **Tampering:** change data in the database; delete transaction records
 - Prevent: Tighter Access Controls, Auditing, Monitoring, Backups

MySQL Database Hardening Best Practices

Installation

- Keep MySQL up to date
- MySQL Installer for Windows
- Yum/Apt/other Repositories

Authentication

- Password Policies
- Multi factor authentication
- External Authentication
- X.509

Authorization

- Remove Extra Accounts
- Grant Minimal Privileges
- Audit users and privileges

Configuration

- Firewall
- Auditing and Logging
- Limit Network Access
- Monitor changes

Encryption

- SSL/TLS for Secure Connections
- Data Encryption (AES, RSA)
- TDE
- Masking and De-Identification

Collateral

- Encrypt Backups
- Secure Replications
- Secure HA and DR

Installation

Hardening: Database and OS Installation, Configuration & Maintenance

- Build on a secure platform
 - OS patched, firewall, etc.
- Use the Secure build
 - <https://dev.mysql.com/doc/mysql-secure-deployment-guide/8.0/en/>
 - <https://dev.mysql.com/doc/mysql-secure-deployment-guide/5.7/en/>
- Keep **Operating System** and **MySQL security patches up to date**
 - May require a restart (MySQL or operating system) to take effect
- Follow OS vendor specific hardening guidelines
 - For example Tips for Hardening an Oracle Linux Server:
<http://www.oracle.com/technetwork/articles/servers-storage-admin/tips-harden-oracle-linux-1695888.html>
- Alternatively follow CIS Benchmark guides

Authentication

MySQL User Accounts

- Within MySQL, accounts are composed by three elements:
 - username@host + password
- Username is a couple of values: username and a client (host)
 - "host" is the reference of the machine where the mysql client run ('%' means "all the IPs")
 - Host may use wildcards (e.g. '%' means every host)
 - Specify username only, means "every host"

```
'user_name'@'myhost.oracle.com'
```

```
'db_user'@'192.168.1.1'
```

- Blank usernames are considered anonymous
 - ''@'localhost'
- Example

```
Create user 'test'@'%' identified by 'VeryComplex1!';
```

Users password

- The default Authentication plugin in 8.0 is `caching_sha2_password`
 - the MySQL 5.7 default is `mysql_native_password` (it uses the SHA1 algorithm which NIST no longer recommend)
- To change the default plugin
 - Add in the configuration file `my.cnf`, under `[mysqld]` section, for example
`default_authentication_plugin=mysql_native_password`
 - Specify at command line when you invoke the server, for example
`--default_authentication_plugin=sha256_password`

- examples

```
mysql> CREATE USER 'jeffrey'@'localhost' IDENTIFIED BY 'password_value';
mysql> SET PASSWORD FOR jeffrey'@'localhost' = PASSWORD('password_value');
mysql> SET PASSWORD = PASSWORD('mypass');
mysql> CREATE fred@localhost IDENTIFIED WITH mysql_native_password BY 'HisPa55w0rd!';
```

Table Example - mysql.user

- User table contains one row for each account

```
mysql> select user, host, authentication_string from mysql.user;
```

user	host	authentication_string
root	localhost	*BF6F715A6EBF332CCB67E7608EBE63005BEB705C
root	127.0.0.1	*BF6F715A6EBF332CCB67E7608EBE63005BEB705C
root	192.168.1.2	*93A2D6F69B4B1384D9B7E537330DB295343CBB9C
replicate	192.168.1.3	*2A9C19E10B0BCA2309BFA44B71BE75D5740E4A9C

```
4 rows in set (0.00 sec)
```


Password Validation Plugin

- Check robustness of password and prevent usage of weak ones
- It use preconfigured or customized policies (`validate_password_policy` variable)
 - Low: Length
 - Medium: Length; numeric, lowercase/uppercase, and special characters
 - Strong: Length; numeric, lowercase/uppercase, and special characters; dictionary file

- Passwords specified as a hash are not checked

```
mysql> SET PASSWORD = '*0D3CED9BEC10A777AEC23CCC353A8C08A633045E';
```

- Variables:
 - `validate_password_dictionary_file`
 - `validate_password_length`
 - `validate_password_mixed_case_count`, `validate_password_number_count`, `validate_password_special_char_count`

- examples

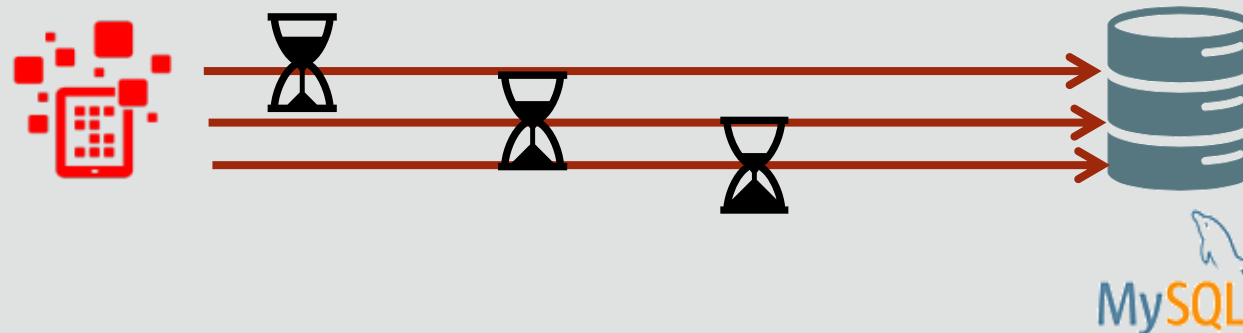
```
mysql> SET PASSWORD = PASSWORD('abc');  
ERROR 1819 (HY000): Your password does not satisfy the current policy
```

Advanced Password Management

- **Password expiration**
 - To require passwords to be changed periodically
- **Password reuse restrictions**
 - To prevent old passwords from being chosen again
- **Password verification**
 - To require that password changes also specify the current password to be replaced
- **Dual passwords**
 - To enable clients to connect using either a primary or secondary password
- **Password strength assessment**
 - To require strong passwords. Implemented using the **password validation component**
- **Random password generation**
 - An alternative to requiring explicit administrator-specified literal passwords
- **Password failure tracking**
 - To enable temporary account locking after too many consecutive incorrect-password login failures

Connection Control Plugin

- Protect from brute force attacks
 - Introduce an increasing delay in server response to clients after a certain number of consecutive failed connection attempts
- CONNECTION_CONTROL plugin
 - checks incoming connections and adds a delay to server responses as necessary
- CONNECTION_CONTROL_FAILED_LOGIN_ATTEMPTS plugin
 - implements an INFORMATION_SCHEMA table that exposes more detailed monitoring information for failed connection attempts



mysql_config_editor

- The mysql_config_editor utility enables you to store authentication credentials in an obfuscated login path file named .mylogin.cnf
 - Useful for scripts and automation
 - Enabled by all the mysql utilities and clients
- mylogin.cnf login path file consists of option groups, each one with host, user, password, port and socket

```
[client]
user = mydefaultname
password = mydefaultpass
host = 127.0.0.1
[mypath]
...
```

- Integrated with MySQL Shell
- Usage example

```
mysql_config_editor set --login-path=client --host=localhost --user=localuser -password
mysql --login-path=mypath
```

<https://dev.mysql.com/doc/refman/8.0/en/mysql-config-editor.html>
<https://dev.mysql.com/doc/mysql-shell/8.0/en/mysql-shell-pluggable-password-store.html>

MySQL Enterprise Authentication

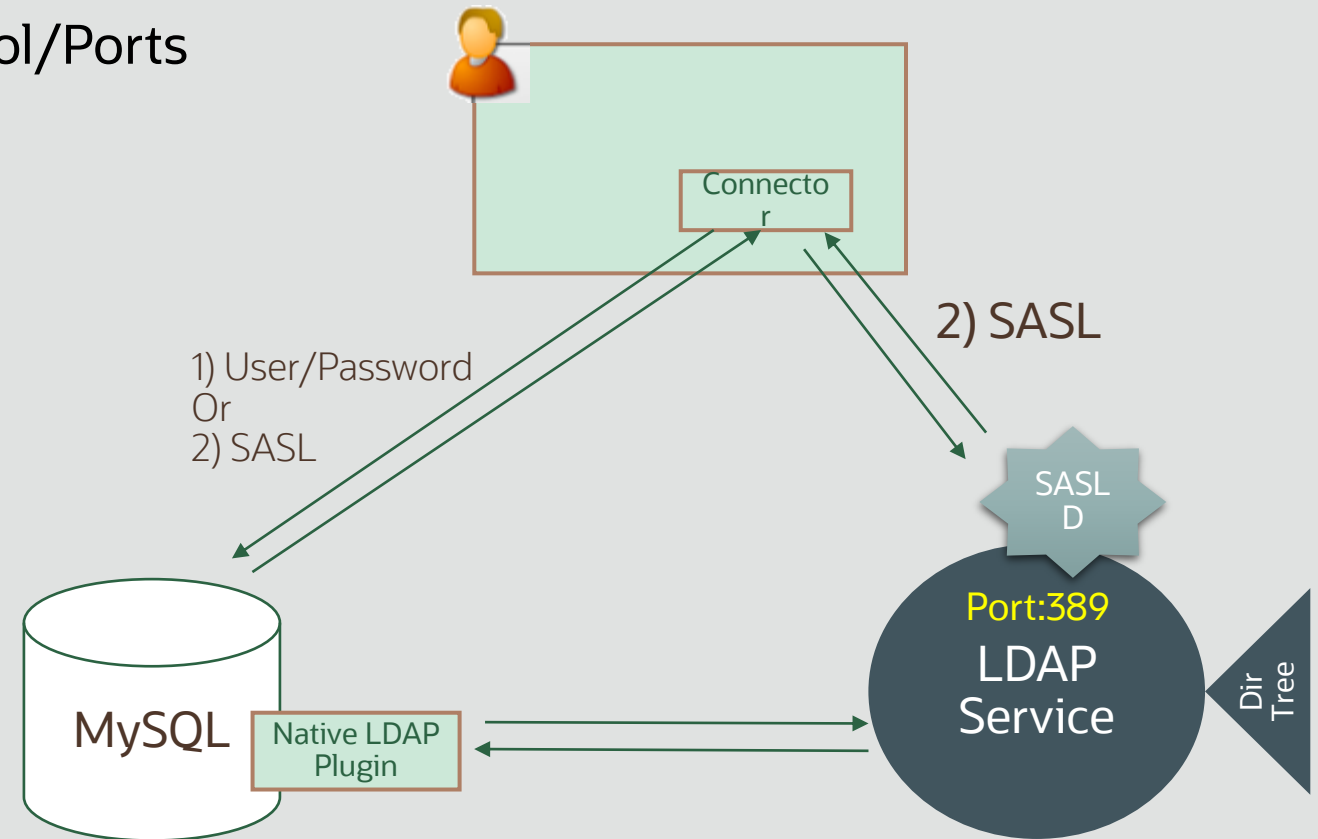
- Simplify your User management, reuse IT passwords
- Integrate with Centralized Authentication Infrastructure
 - Centralized Account Management
 - Password Policy Management
 - Groups & Roles
- Pluggable Authentication Modules (PAM)
 - Linux PAM Standard interface (Unix, LDAP, Kerberos, others)
- Kerberos
- FIDO interface (password less authentication)
- Single Sign On (SSO)
 - Plug-in available to access native LDAP service for authentication
 - Plug-in available to access native Windows service



Integrates MySQL with existing security infrastructures

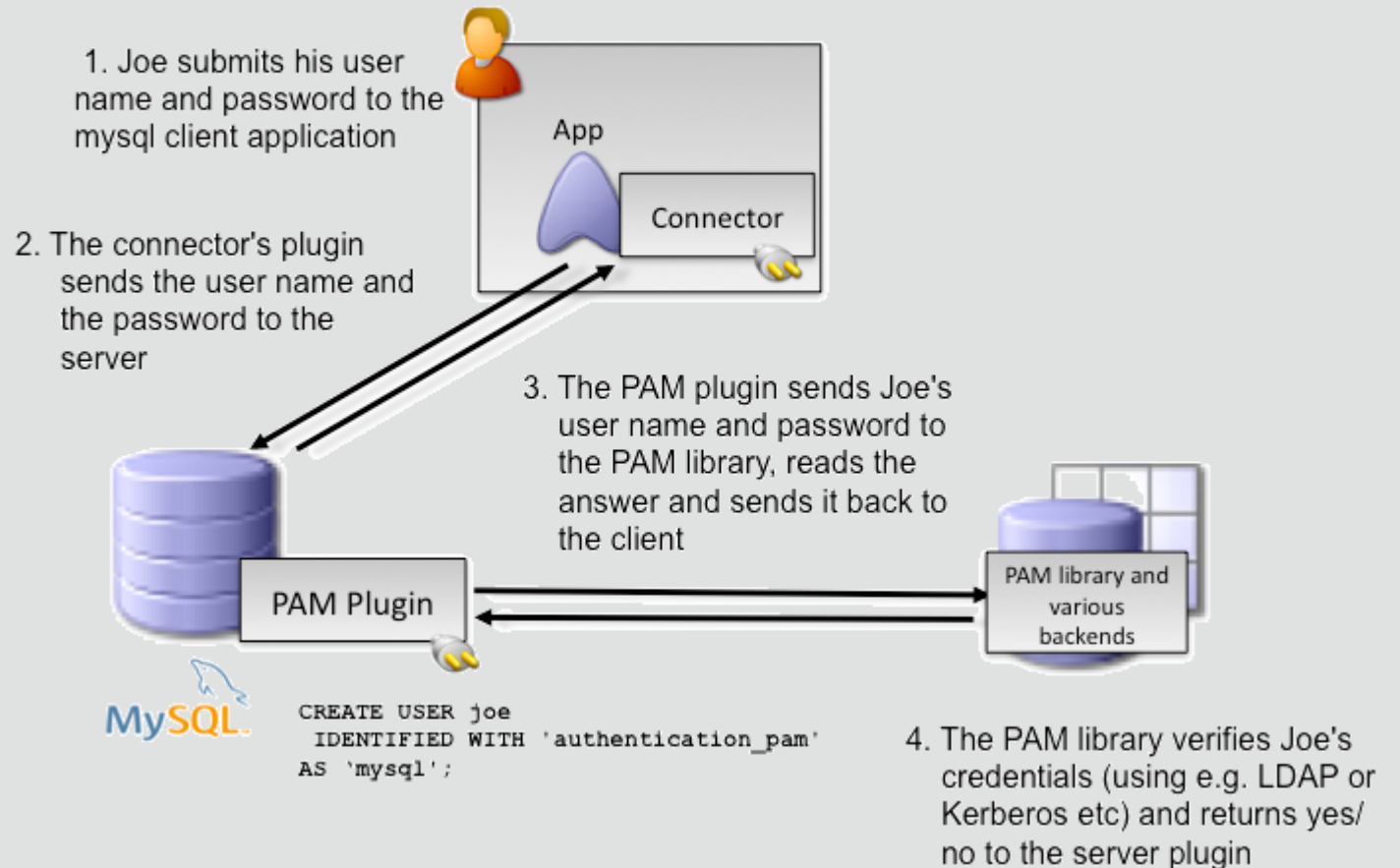
MySQL Enterprise Authentication: Native LDAP

- Direct Connection over LDAP Protocol/Ports
- Authentication with
 - User and Password
 - or SASL
- Customizable for users and groups



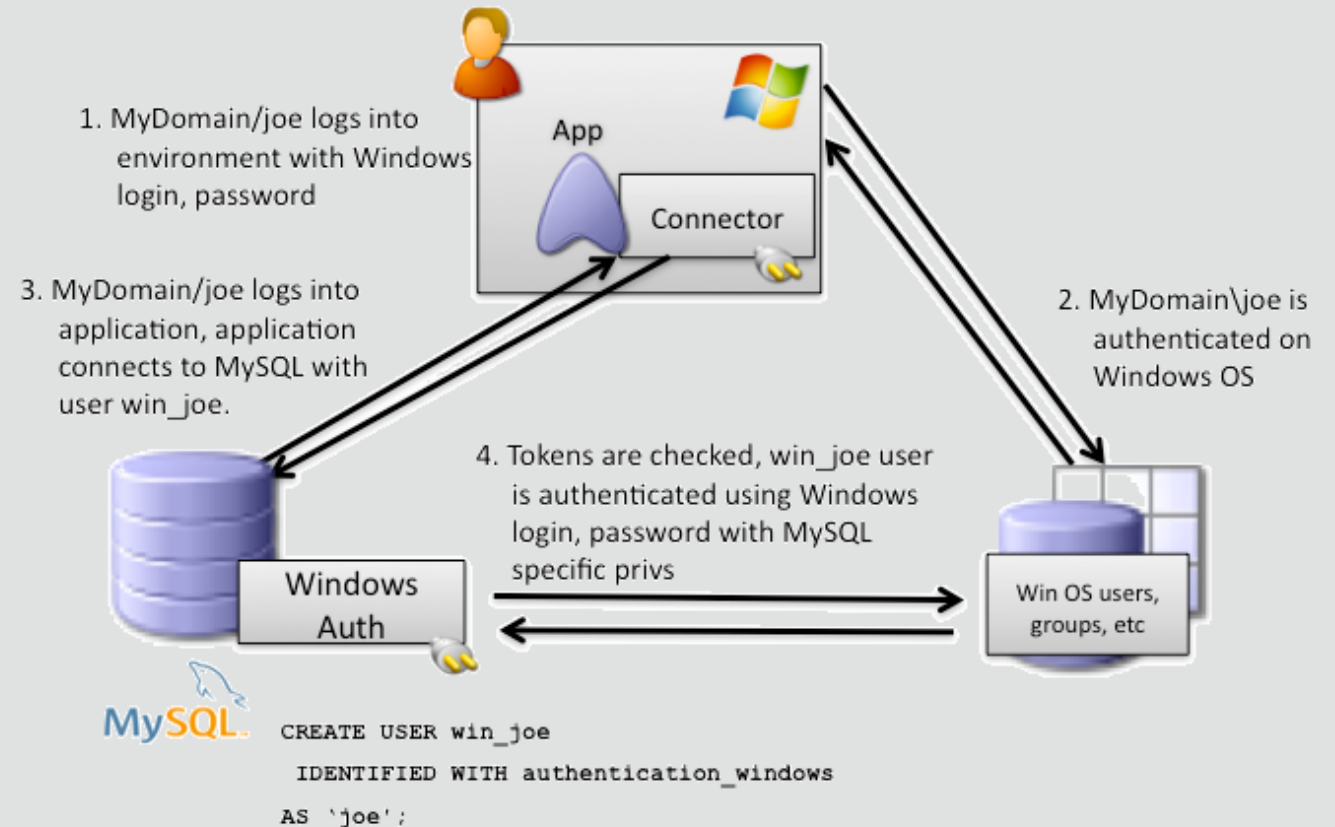
MySQL Enterprise Authentication: PAM

- Standard Interface
 - LDAP
 - Unix/Linux
- Proxy Users



MySQL Enterprise Authentication: **Windows**

- Windows Active Directory
- Windows Native Services
- Authentication handshake
 - Creates a security context object
 - Represents the identity of the Windows client



Example: Windows Native Authentication Plugin

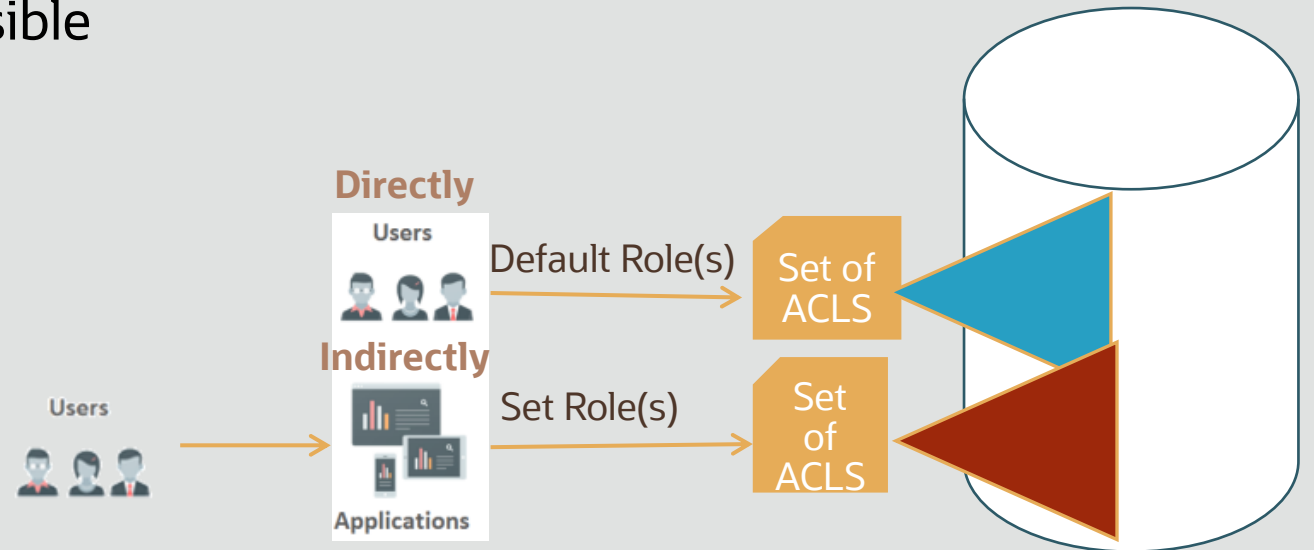
- The MySQL pluggable authentication plug-in for Windows is enabled and configured
- The following command is executed on a MySQL Server:

```
CREATE USER `SQL_admin`@`%` IDENTIFIED WITH authentication_windows  
AS 'joe,henry,"Power Users"';
```

- Discussion
 - User Joe in Windows (AD connected) connect to MySQL ... (ID/PW?)
 - User Carsten in Windows (AD connected) connect to MySQL...
 - User Henry (MAC) connect to MySQL ... (ID/PW?)
 - User Henry (Linux) connect to MySQL ... (ID/PW?)

MySQL Roles

- Improving MySQL Access Controls
- Introduced in the 8.0.0 DMR
- Easier to manage user and applications rights
- As standards compliant as practically possible
- Multiple default roles
- Can export the role in graph in GraphML

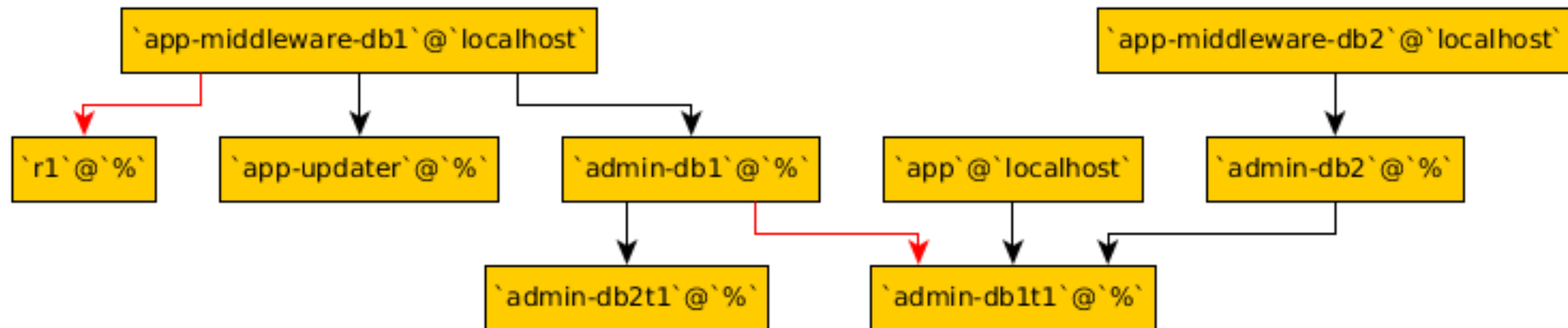


SQL Roles Implementation In a Nutshell

- A role is basically a user account with login disabled
- A memory based hash of flattened privilege sets for each active role
- Operations
 - 2 new SQL functions: `CURRENT_ROLE()` and `ROLE_GRAPHML()`
 - 3 new global privileges: `CREATE ROLE`, `DROP ROLE` and `ROLE_ADMIN`
 - Extensions to: `ALTER USER`, `GRANT/REVOKE`, `SET` and `SHOW GRANTS`
- Assignments
 - Pre-roles ACL code is used when there's no active role(s)
 - Users can be assigned several roles
 - Users can have zero or more default roles
 - Active Roles can be changed (from various assigned roles)
 - For example just escalate or change privileges from within an application for certain operations

Role Examples

```
1 CREATE ROLE `r1`, `admin-db1`, `admin-db2`, `admin-db1t1`, `admin-db2t1`, `app-updater`;
2
3 CREATE USER `app-middleware-db1`@localhost, `app-middleware-db2`@localhost, `app`@localhost;
4
5 GRANT `admin-db1`, `app-updater` TO `app-middleware-db1`@localhost;
6 GRANT `r1` TO `app-middleware-db1`@localhost WITH ADMIN OPTION;
7 GRANT `admin-db2t1` TO `admin-db1`;
8 GRANT `admin-db1t1` TO `admin-db1` WITH ADMIN OPTION;
9 GRANT `admin-db1t1` TO `admin-db2`, `app`@localhost;
10 GRANT `admin-db2` TO `app-middleware-db2`@localhost;
```



Authorization

MySQL Privileges

- Grant X privileges on Y to Z [with grant options]

- Administrative Privileges: Global, enable users to manage the server
- Database Privileges: Specific to database(s)

Grant all privileges on *.* TO 'test'@'%' with grant option;

- Fine grained controls over user privileges

- Creating, altering and deleting databases
- Creating, altering and deleting tables; create or delete indexes
- Create, execute, or delete stored procedures and with what rights
- Execute INSERT, SELECT, UPDATE, DELETE queries

- Deny access by user name is not possible

- but you can disable a user

- Control can be disabled with --skip-grant-tables

- Evaluate skip_networking to use in combination
- <https://dev.mysql.com/doc/refman/8.0/en/resetting-permissions.html>

Privilege	Meaning and Grantable Levels
<u>ALL</u> [<u>PRIVILEGES</u>]	Grant all privileges at specified access level except <u>GRANT OPTION</u> and <u>PROXY</u> .
<u>ALTER</u>	Enable use of <u>ALTER TABLE</u> . Levels: Global, database, table.
<u>ALTER ROUTINE</u>	Enable stored routines to be altered or dropped. Levels: Global, database, procedure.
<u>CREATE</u>	Enable database and table creation. Levels: Global, database, table.
<u>CREATE ROUTINE</u>	Enable stored routine creation. Levels: Global, database.
<u>CREATE TABLESPACE</u>	Enable tablespaces and log file groups to be created, altered, or dropped. Level: Global.
<u>CREATE TEMPORARY TABLES</u>	Enable use of <u>CREATE TEMPORARY TABLE</u> . Levels: Global, database.
<u>CREATE USER</u>	Enable use of <u>CREATE USER</u> , <u>DROP USER</u> , <u>RENAME USER</u> , and <u>REVOKE ALL PRIVILEGES</u> . Level: Global.
<u>CREATE VIEW</u>	Enable views to be created or altered. Levels: Global, database, table.
<u>DELETE</u>	Enable use of <u>DELETE</u> . Level: Global, database, table.
<u>DROP</u>	Enable databases, tables, and views to be dropped. Levels: Global, database, table.
<u>EVENT</u>	Enable use of events for the Event Scheduler. Levels: Global, database.
<u>EXECUTE</u>	Enable the user to execute stored routines. Levels: Global, database, table.
<u>FILE</u>	Enable the user to cause the server to read or write files. Level: Global.
<u>GRANT OPTION</u>	Enable privileges to be granted to or removed from other accounts. Levels: Global, database, table, procedure, proxy.
<u>INDEX</u>	Enable indexes to be created or dropped. Levels: Global, database, table.
<u>INSERT</u>	Enable use of <u>INSERT</u> . Levels: Global, database, table, column.
<u>LOCK TABLES</u>	Enable use of <u>LOCK TABLES</u> on tables for which you have the <u>SELECT</u> privilege. Levels: Global, database.
<u>PROCESS</u>	Enable the user to see all processes with <u>SHOW PROCESSLIST</u> . Level: Global.
<u>PROXY</u>	Enable user proxying. Level: From user to user.
<u>REFERENCES</u>	Enable foreign key creation. Levels: Global, database, table, column.
<u>RELOAD</u>	Enable use of <u>FLUSH</u> operations. Level: Global.
<u>REPLICATION CLIENT</u>	Enable the user to ask where master or slave servers are. Level: Global.
<u>REPLICATION SLAVE</u>	Enable replication slaves to read binary log events from the master. Level: Global.
<u>SELECT</u>	Enable use of <u>SELECT</u> . Levels: Global, database, table, column.
<u>SHOW DATABASES</u>	Enable <u>SHOW DATABASES</u> to show all databases. Level: Global.
<u>SHOW VIEW</u>	Enable use of <u>SHOW CREATE VIEW</u> . Levels: Global, database, table.
<u>SHUTDOWN</u>	Enable use of <u>mysqladmin shutdown</u> . Level: Global.
<u>SUPER</u>	Enable use of other administrative operations such as <u>CHANGE MASTER TO</u> , <u>KILL</u> , <u>PURGE BINARY LOGS</u> , <u>SET GLOBAL</u> , and <u>RESET SLAVE</u> .
<u>TRIGGER</u>	Enable trigger operations. Levels: Global, database, table.
<u>UPDATE</u>	Enable use of <u>UPDATE</u> . Levels: Global, database, table, column.
<u>USAGE</u>	Synonym for "no privileges"



When Privilege Changes Take Effect

- Privileges are stored in tables
 - user: User accounts, global privileges, and other non-privilege columns
 - db: Database-level privileges
 - tables_priv: Table-level privileges
 - columns_priv: Column-level privileges
 - procs_priv: Stored procedure and function privileges
 - proxies_priv: Proxy-user privileges
- Manual changes - INSERT, UPDATE, or DELETE - requires reload of grant tables
 - `mysql> FLUSH PRIVILEGES`
 - `# mysqladmin flush-privileges`
 - `# mysqladmin reload`

Dynamic Privileges

- Provides finer grained administrative level access controls
- Too often SUPER is required for tasks when less privilege is really needed
 - Support concept of “least privilege”
- Needed to allow adding administrative access controls
 - Now can come with new components
 - Examples: Replication, HA , Backup
- Uses the familiar
 - `GRANT <dynamic_acl> ON *.* TO ... syntax`

User Roles improve Access Control

- **Easier to manage user and application privileges. The process**
 - Create a role (e.g. app_developer, app_user) then assign privileges to the role (using GRANT)
 - Assign the role to a user, e.g. GRANT app_developer TO stuart@localhost;
 - A user may be assigned more than one role
 - To specify which roles become active each time a user connects, use SET DEFAULT ROLE, otherwise use SET ROLE to activate on a per session basis
 - To check roles/privs use: SHOW GRANTS FOR <user>; SHOW GRANTS FOR <user> USING <role>;
 - Role privileges and roles can be removed used REVOKE syntax; roles can also be dropped
- **Mandatory roles for all users can be set at a server level**
 - Either in my.cnf: mandatory_roles='role1,role2,roleN'
 - Or in sql: SET PERSIST mandatory_roles = 'role1,role2,roleN';

Users and Privileges

- **SYSTEM_USER privilege**
 - Users granted 'SYSTEM_USER' privilege can be seen as power users
 - Enforces two categories of users, power and regular
 - Regular users can not modify any properties of power users – even if DDL privileges are granted
 - Provides strong separation between administrators and those that are using the environment
 - https://dev.mysql.com/doc/refman/8.0/en/privileges-provided.html#priv_system-user
- **CONNECTION_ADMIN privilege**
 - Enables use of the KILL statement or mysqladmin kill command to kill threads belonging to other accounts. (An account can always kill its own threads.)
- **Partial Revokes**
 - Enables privileges for specific schemas to be revoked while leaving the privileges in place for other schemas
 - DBA can restrict DDL/DML operations to specific databases
 - Even if the user has the required global privileges; creates “schema” administration only limits – no instance wide rights



Hands-On Labs



4a. Users Management

4b. MySQL Roles

Configuration

MySQL Database Hardening: Configuration

- **Audit Activity**
 - Use Enterprise Audit
 - Alt. Transiently enable Query Logging
 - Monitor and Inspect regularly
- **Disable or Limit Remote Access**
 - If local “skip-networking” or bind-address=127.0.0.1
 - If Remote access then limit hosts/IP
- **Consider changing default port**
- **Change root username**
- **Disable unauthorized reading from local files**
 - Disable LOAD DATA LOCAL INFILE
- **Run MySQL on non default port**
 - More difficult to find database
- **Limit MySQL OS User**
- **Configure External Key Management**

MySQL Enterprise Monitor

- Enforce MySQL Security Best Practices
 - Identify vulnerabilities
- Easily ensure all your MySQL assets are hardened and secure
 - User & Password Monitoring
 - Schema Change Monitoring
 - Configuration Management
- Manage / Configure
 - MySQL Enterprise Firewall
 - MySQL Enterprise Audit
 - MySQL Enterprise Backup
- ...And then there's its day job
 - Alerting when thresholds are breached
 - Assisting with troubleshooting using query analyzer

	Account Has Old Insecure Password Hash
	Account Requires Unavailable Authentication Plugins
	Insecure Password Authentication Option Is Enabled
	Insecure Password Generation Option Is Enabled
	LOCAL Option Of LOAD DATA Statement Is Enabled
	MySQL Enterprise Audit Plugin
	MySQL Enterprise Firewall
	MySQL User Account Advisor
	Policy-Based Password Validation Does Not Perform Dictionary Checks
	Policy-Based Password Validation Is Weak
	Policy-Based Password Validation Not Enabled
	Privilege Alterations Detected: Privileges Granted
	Privilege Alterations Detected: Privileges Revoked
	Privilege Alterations Have Been Detected
	Root Account Can Login Remotely
	Root Account Without Password
	SHA-256 Password Authentication Not Enabled
	Server Contains Default "test" Database
	Server Has Accounts Without A Password
	Server Has Anonymous Accounts Not Being Used As Proxies
	Server Has No Locally Authenticated Root User
	Server Includes A Root User Account
	Symlinks Are Enabled
	User Has Rights To Database That Does Not Exist
	User Has Rights To Table That Does Not Exist
	Users Can View All Databases On MySQL Server

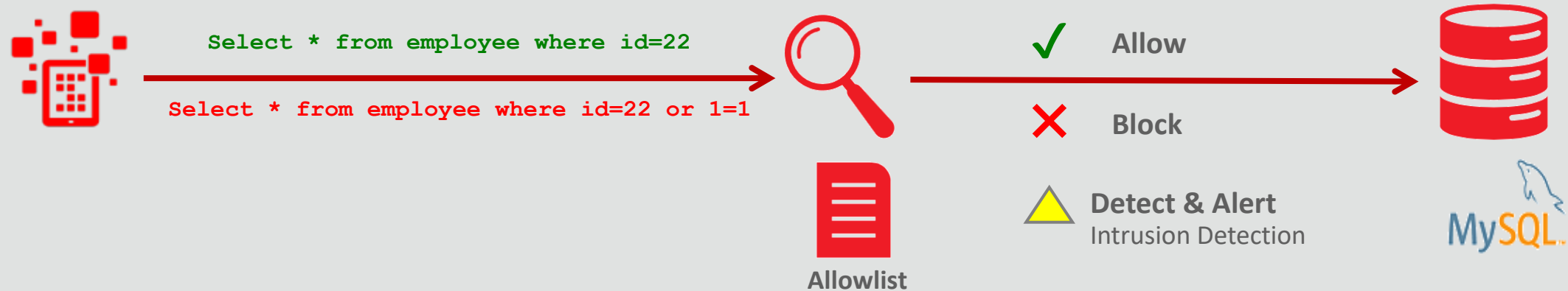
MySQL Enterprise Firewall

- Real Time Protection
 - Queries checked against Allow List and Firewall Rules
- Blocks SQL Injection Attacks
 - Block Out of Policy Transactions
- Intrusion Detection
 - Detect and Alert on Out of Policy Transactions
- Automated Allow List
 - Learning mode for creation of approved list of SQL command patterns on a per user or profile (multiple users) basis
- Transparent
 - No changes to application required
- Combined Firewall/Audit Rules
 - Create more general allow/deny firewall rules using JSON syntax

Enterprise Firewall			Configured: 8 of 8
☐ Item			Info
☒  Account Has Overly Permissive White List			
☒  Account Sending Excessive Percentage of Blocked Queries			
☒  Account Without Firewall Protection			
☒  Excessive Number of Queries Blocked By Firewall			
☒  Firewall Max Query Size Too Small			
☒  Firewall Not Enabled			
☒  Firewall Not Installed			
☒  Firewall Trace Has Been Enabled			

MySQL Enterprise Firewall monitoring

MySQL Enterprise Firewall



- Use it to Block SQL Injection Attacks
 - Allow: SQL Statements that match Whitelist
 - Block: SQL statements that are not on Whitelist
- Use it as an Intrusion Detection System
 - Detect: SQL statements that are not on Allowlist
 - SQL Statements execute and alert administrators

Firewall Group and Account profiles

- **Account profiles:** Allowlist applies only to a single account (deprecated)
- **Group profiles:** Allowlist applies when the session account matches any account that is a member of the group
 - A group with a single account is equivalent to an account profile
- **New privileges:** FIREWALL_ADMIN, FIREWALL_EXEMPT, FIREWALL_USER, EXECUTE
 - FIREWALL_ADMIN is not automatically assigned to root !
- **Group profiles give you more flexibility**
 - A group profile can include multiple accounts as members
 - An account can be a member of multiple group profiles
 - Multiple profiles can match a given client
- **In both cases, rules are normalized**

- e.g. these queries

```
SELECT first_name, last_name FROM customer WHERE customer_id = 1;  
select first_name, last_name from customer where customer_id = 99;  
SELECT first_name, last_name FROM customer WHERE customer_id = 143;
```

- Are written as

```
SELECT `first_name` , `last_name` FROM `customer` WHERE `customer_id` = ?
```

MySQL Enterprise Firewall - Example 1

Account profile

- Recording Mode

```
mysql> CALL mysql.sp_set_firewall_mode('fwuser@localhost', 'RECORDING');
```

```
mysql> SELECT first_name, last_name FROM customer WHERE customer_id=1;
```

- Protection Mode

```
mysql> CALL mysql.sp_set_firewall_mode('fwuser@localhost', 'PROTECTING');
```

```
mysql> SELECT first_name, last_name FROM customer WHERE customer_id='48'; (success)
```

```
mysql> SELECT first_name, last_name FROM customer WHERE customer_id='92'; (success)
```

```
mysql> SELECT first_name, last_name FROM customer WHERE customer_id = 1 OR TRUE; (failure)
```

```
ERROR 1045 (28000): Statement was blocked by Firewall
```

- Interrogate status

```
mysql> SHOW GLOBAL STATUS LIKE 'firewall%';
```

```
mysql> SELECT MODE FROM INFORMATION_SCHEMA.MYSQL_FIREWALL_USERS WHERE USERHOST = 'fwuser@localhost';
```

```
mysql> SELECT RULE FROM INFORMATION_SCHEMA.MYSQL_FIREWALL_WHITELIST WHERE USERHOST = 'fwuser@localhost';
```

```
mysql> SHOW GLOBAL STATUS LIKE 'Firewall%';
```

Variable_name	Value
Firewall_access_denied	3
Firewall_access_granted	2
Firewall_access_suspicious	1
Firewall_cached_entries	4

MySQL Enterprise Firewall - Example 2

Group profile

- Create a new group and assign a member

```
mysql> GRANT FIREWALL_ADMIN on *.* to root@localhost;
mysql> CALL mysql.sp_set_firewall_group_mode('fwgrp', 'RECORDING');
mysql> CALL mysql.sp_firewall_group_enlist('fwgrp', 'fwtest@%');
```

- Protection Mode

```
mysql> CALL mysql.sp_set_firewall_group_mode('fwgrp', 'PROTECTING');
mysql> SELECT first_name, last_name FROM customer WHERE customer_id='48'; (success)
mysql> SELECT first_name, last_name FROM customer WHERE customer_id='92'; (success)
mysql> SELECT first_name, last_name FROM customer WHERE customer_id = 1 OR TRUE; (failure)
ERROR 1045 (28000): Statement was blocked by Firewall
```

- Interrogate status

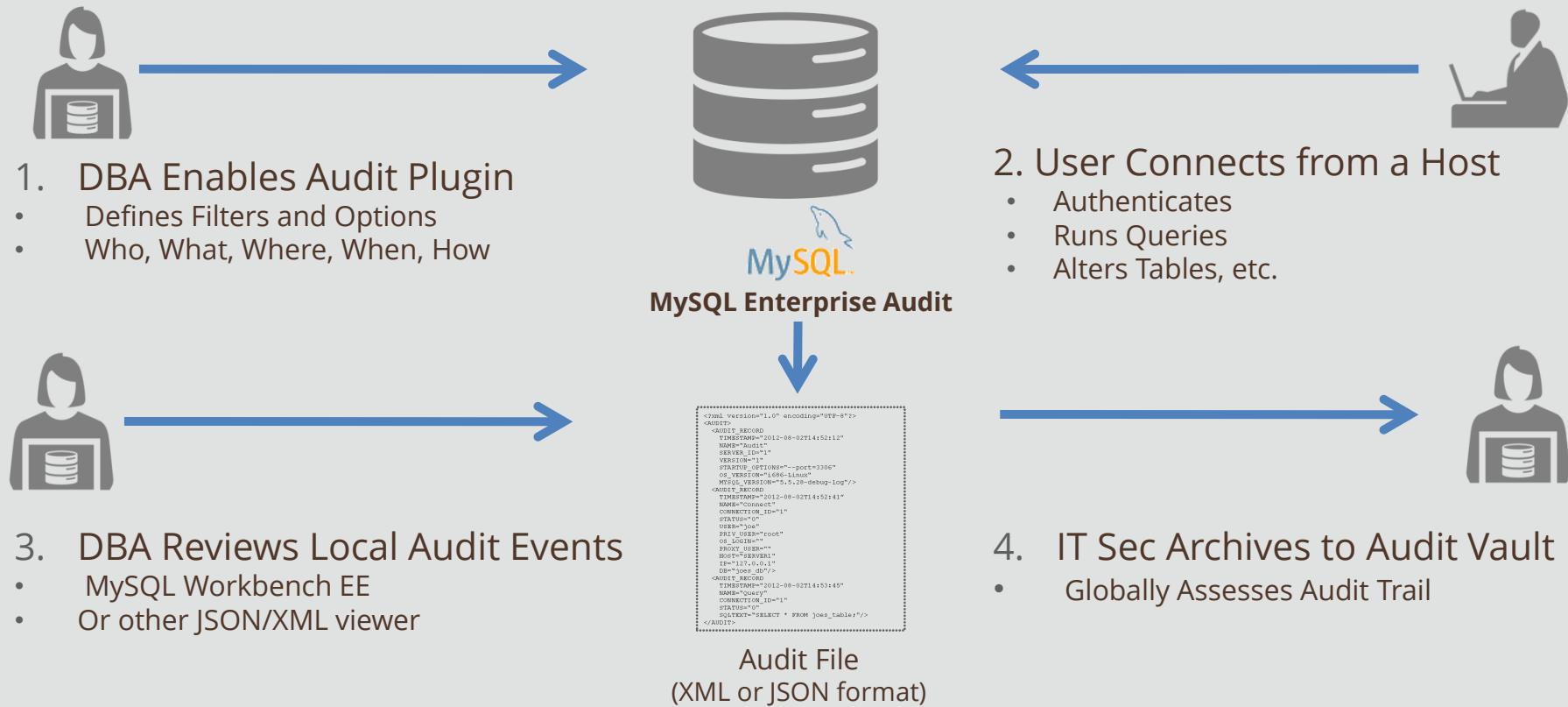
```
mysql> SHOW GLOBAL STATUS LIKE 'firewall%';
mysql> SELECT MODE FROM performance_schema.firewall_groups WHERE NAME = 'fwgrp';
mysql> SELECT * FROM performance_schema.firewall_membership WHERE GROUP_ID = 'fwgrp';
mysql> SELECT RULE FROM performance_schema.firewall_group_allowlist WHERE NAME = 'fwgrp';
```

MySQL Enterprise **Audit**

- Out-of-the-box logging of connections, logins, and query
 - Auditing for Security & Compliance: GDPR, PCI DSS, Sarbanes-Oxley, HIPAA, FIPS, ...
 - You see who has done what - connections, logins, and queries
- Secure SQL Access to audit events
- Simple to fine grained policies for filtering, and log rotation
- Dynamically enabled, disabled: no server restart
- XML and JSON audit stream formatting options
- Compression & Encryption

Adds regulatory compliance to MySQL
applications
(GDPR, HIPAA, PCI DSS, etc.)

MySQL Enterprise Audit workflow



Audit Filtering

- Allows DBAs to “custom” design audit process
- Use very fine grained rules
 - Reduce audit log file size
 - Reduce File System IO and Storage / Increases performance (less items logged)
 - Increases audit log post processing efficiency – less data to process for immediate answers
 - Defined using JSON
- Complete Audit Data:
 - Who, What, When, How, Status
 - From Where
 - DB version, OS version, Options
 - And more

```
<?xml version="1.0" encoding="UTF-8"?>
<AUDIT>
  <AUDIT_RECORD
    TIMESTAMP="2012-08-02T14:52:12"
    NAME="Audit"
    SERVER_ID="1"
    VERSION="1"
    STARTUP_OPTIONS="--port=3306"
    OS_VERSION="i686-Linux"
    MYSQL_VERSION="5.5.28-debug-log"/>
  <AUDIT_RECORD
    TIMESTAMP="2012-08-02T14:52:41"
    NAME="Connect"
    CONNECTION_ID="1"
    STATUS="0"
    USER="joe"
    PRIV_USER="root"
    OS_LOGIN=""
    PROXY_USER=""
    HOST="SERVER1"
    IP="127.0.0.1"
    DB="joes_db"/>
  <AUDIT_RECORD
    TIMESTAMP="2012-08-02T14:53:45"
    NAME="Query"
    CONNECTION_ID="1"
    STATUS="0"
    SQLTEXT="SELECT * FROM joes_table;"/>
</AUDIT>
```

Encryption

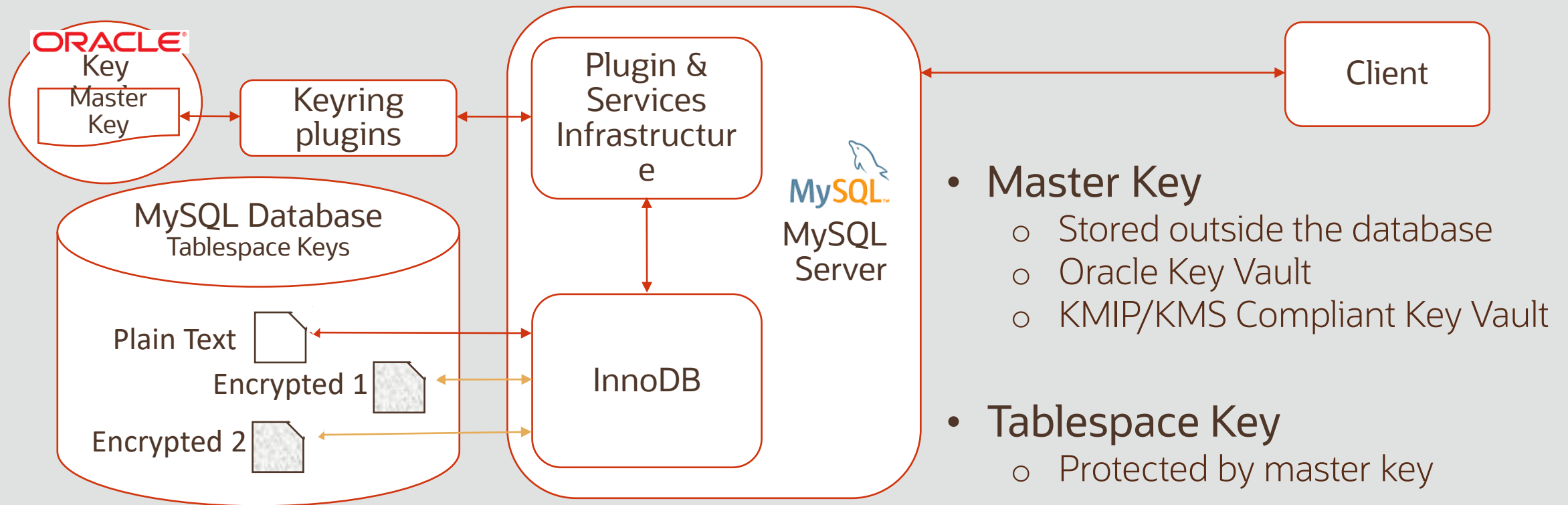
MySQL Enterprise

Transparent Data Encryption (TDE)

- Protects against Attacks on Database Files
- Data at Rest Encryption
 - [System | General | Data Dictionary] Tablespaces, Undo/Redo & Binary/Relay logs,
 - Strong Encryption – AES 256
- Transparent to applications and users
 - No application code, schema or data type changes
- Transparent to DBAs
 - Keys are hidden from DBAs, no configuration changes
- Requires Key Management
 - Protection, rotation, storage, recovery
 - Integrates with KMIP 1.1 and KMS

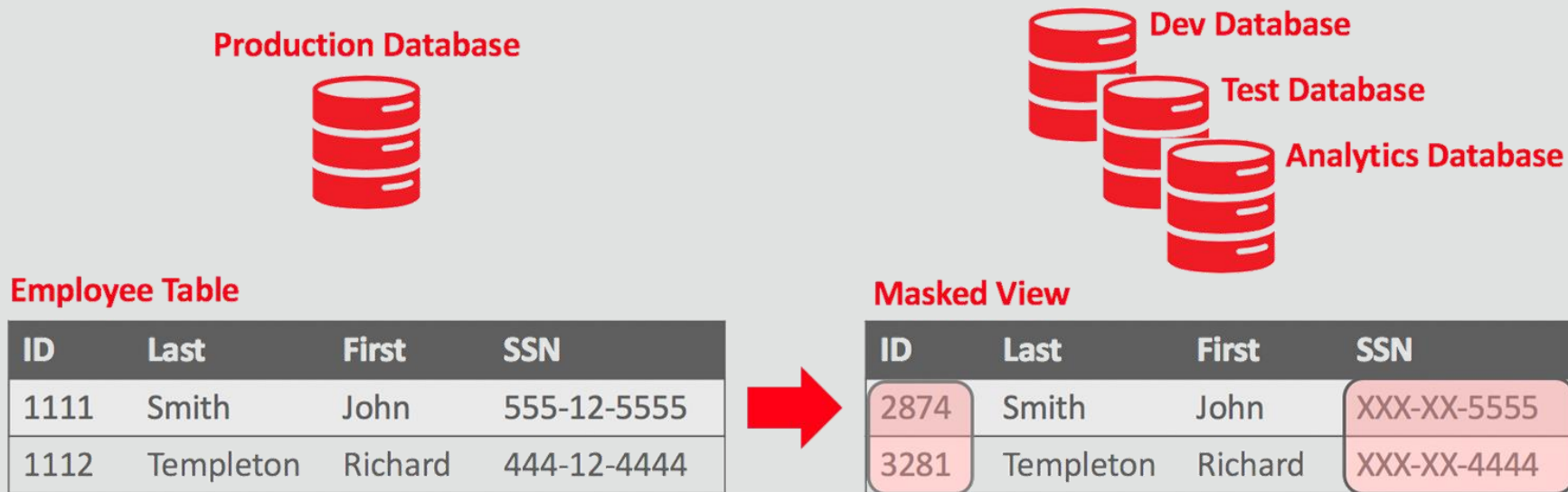


MySQL Enterprise TDE architecture



MySQL Enterprise Masking & De-Identification

- Data De-identification helps database customers **improve security** and helps **accelerate compliance**
- Reduce IT costs by simplifying sanitizing production data
 - Transforming sensitive data for use in analytics, testing, development, and more



MySQL Enterprise Masking & De-Identification

- Data Masking
 - String
 - Dictionary based
 - Specific
 - Social Security Number
 - Payment card (strict/relaxed)
- Random Data Generator
 - Random number within a range
 - Email addresses
 - Payment card (Luhn check compliant)
 - Social Security Number
 - Dictionary based generation

```
UPDATE users SET designation = CAST(gen_dictionary("Designations") as CHAR) where id > 0 ;
UPDATE users SET ssn = CAST(gen_rnd_ssn() as CHAR) where id > 0;
UPDATE users SET salary = gen_range(40000, 55000) WHERE designation in ('Developer') and id > 0;
UPDATE users SET salary = gen_range(60000, 70000) WHERE designation in ('Senior Developer') and id > 0;
UPDATE users SET salary = gen_range(75000, 90000) WHERE designation in ('Principal Developer', 'Senior Developer') and id > 0;
UPDATE users SET salary = gen_range(95000, 120000) WHERE designation in ('Architect', 'Senior Manager') and id > 0;
UPDATE users SET salary = gen_range(125000, 150000) WHERE designation in ('Director') and id > 0;
UPDATE users SET salary = gen_range(160000, 200000) WHERE designation in ('Senior Director') and id > 0;
UPDATE users SET salary = gen_range(220000, 250000) WHERE designation in ('Vice President') and id > 0;
UPDATE users SET email = gen_rnd_email() where id > 0;
UPDATE users SET phone = gen_rnd_us_phone() where id > 0;
UPDATE users SET payment_card = gen_rnd_pan() where id > 0;
```

Final remarks

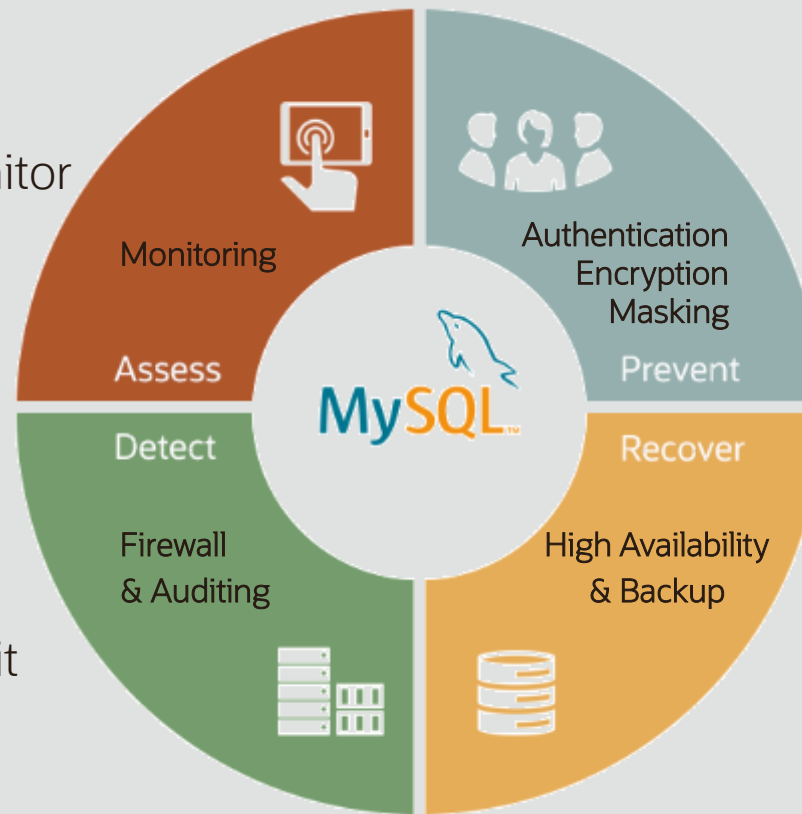
MySQL Enterprise Security

Assess

- MySQL Enterprise Monitor

Detect

- MySQL Enterprise Audit



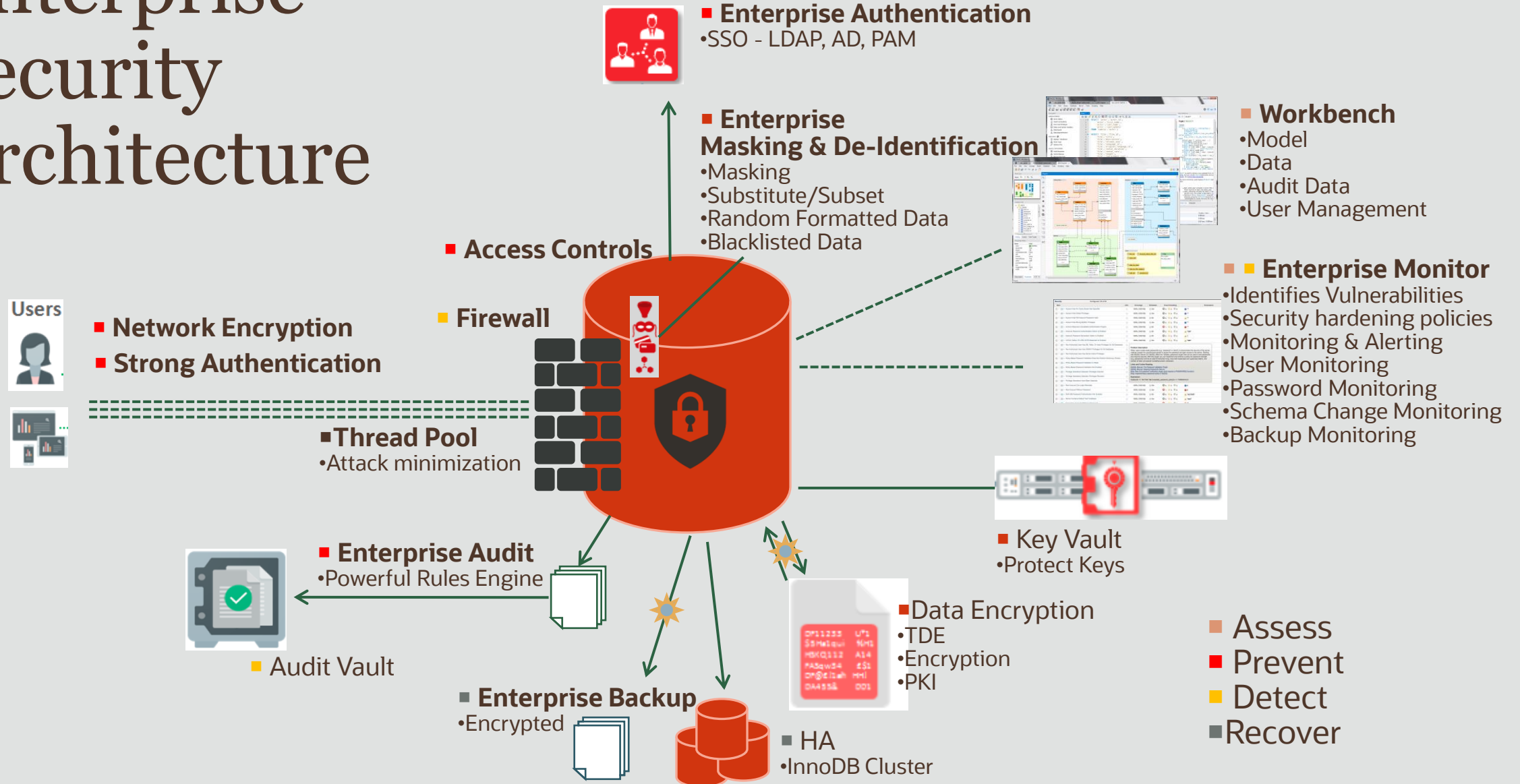
Prevent

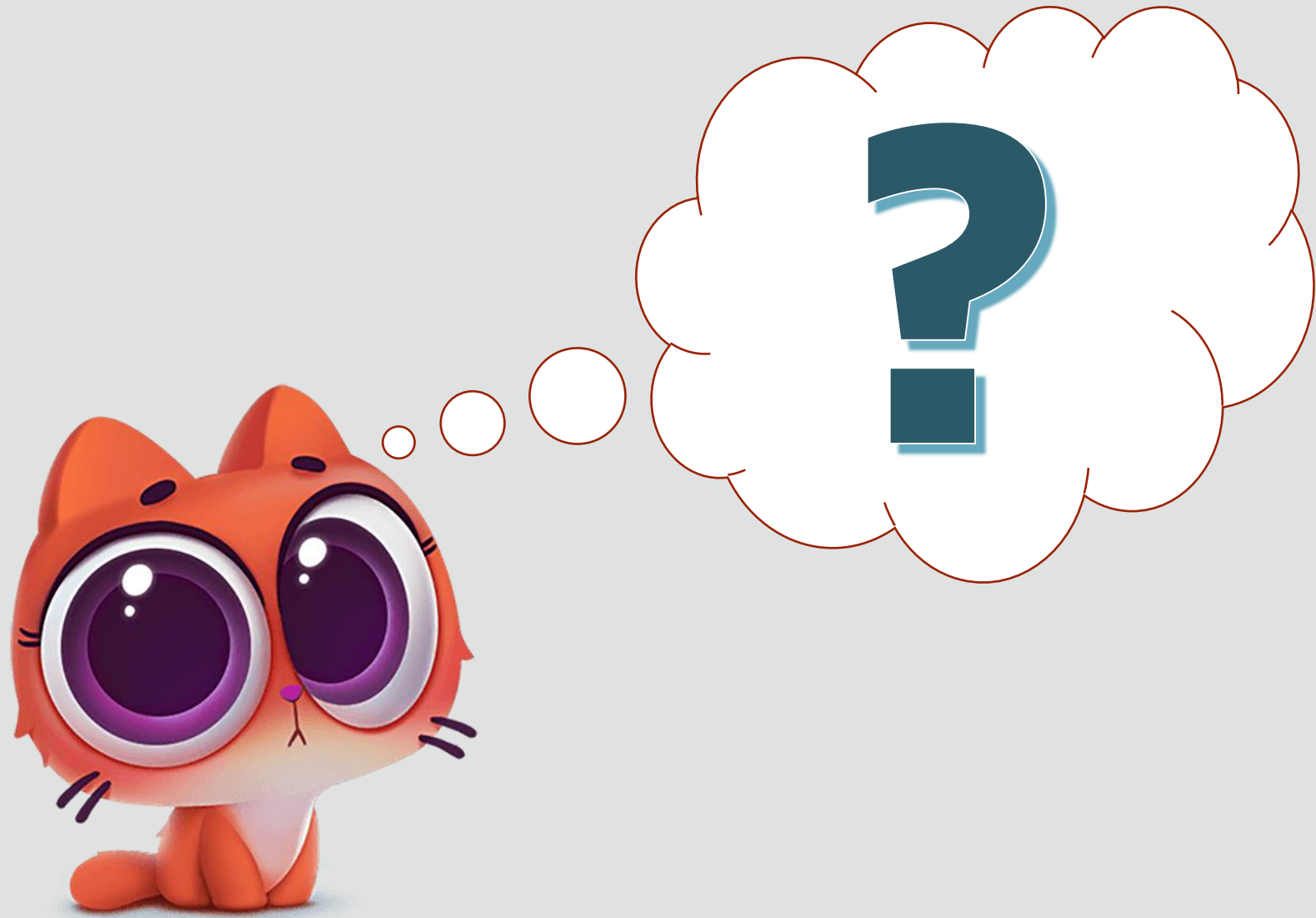
- MySQL Enterprise Authentication
- MySQL Enterprise Firewall
- MySQL Enterprise Encryption
- MySQL Enterprise Data Masking

Recover

- MySQL Enterprise HA
- MySQL Enterprise Backup

Enterprise Security Architecture





Hands-On Labs



4c. MySQL Enterprise Firewall – Account profiles

4d. MySQL Enterprise Firewall – Group profiles

4e. MySQL Enterprise Audit

4f. Data Masking and de-identification



ORACLE